Data-Structures & Algorithms

with

Manoj Prabhakaran

Lecture 18

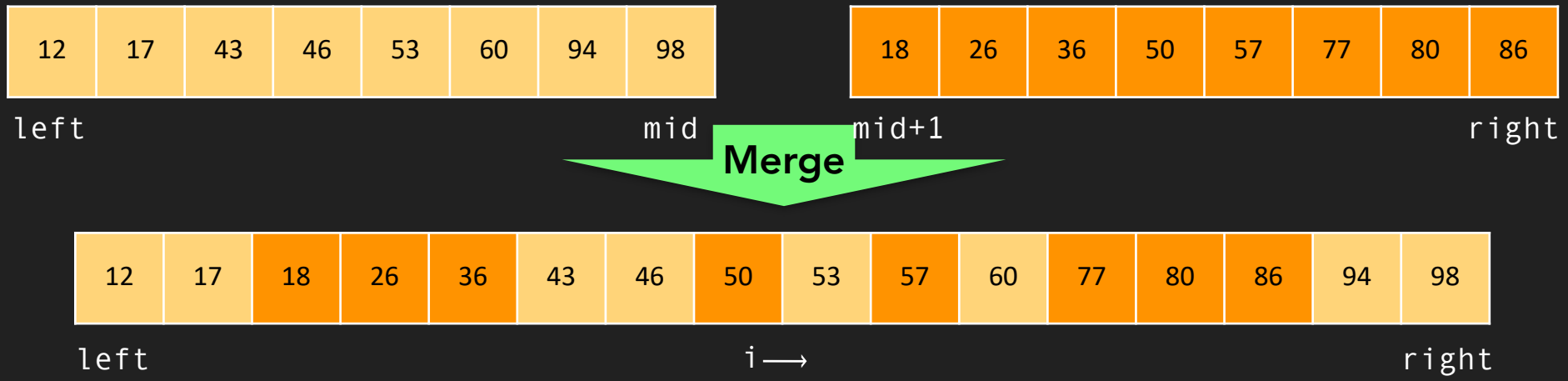
Sorting (ctd.)

Sorting

- So far:
  - Selection sort and Heapsort:
    - $O(n^2)$ vs $O(n \log n)$. Both in-place, not stable.
  - Insertion sort and Tree sort:
    - $O(n^2)$, in-place vs $O(n \log n)$, not in-place. Both stable.
  - Merge sort
    - $O(n \log n)$. Not in-place. Stable.
    - Let's recap

Merge Sort

```
// merge X[left..mid] and X[mid+1..right] into Y[left..right]
void merge(const vector<int>& X, int left, int mid, int right, vector<int>& Y) {
     int L = left, R = mid+1;   // L,R: next indices of left/right halves
     for(int i=left; i <= right; ++i) {
          if(L <= mid && (R > right || X[L] <= X[R])) Y[i] = X[L++];    // copy from left
          else Y[i] = X[R++];                                           // copy from right
     }
}
```



Merge Sort

```
void sort (const std::vector<int>& in, int left,
           int right, std::vector<int>& out,
                      std::vector<int>& aux) {

  if (left == right)
    out[left] = in[left];
  if (left >= right)
    return;

  int mid = (left+right)/2;

  sort(in,left,mid,aux,out);  // outputs into aux
  sort(in,mid+1,right,aux,out);

  merge(aux,left,mid,right,out); // aux is input
}
```

Output will be in `out[left,...,right]`. A temporary vector `aux` is passed (to avoid allocating Θ(n) space in each recursive call)

Total extra space is O(n) + O(log n) = O(n)

- Time?
  - T(n) = 2T(n/2) + O(n)
  - O(n log n)

Merge Sort

```
void sort (const std::vector<int>& in, int left,
           int right, std::vector<int>& out,
                      std::vector<int>& aux) {

  if (left == right)
    out[left] = in[left];
  if (left >= right)
    return;

  int mid = (left+right)/2;

  sort(in,left,mid,aux,out);  // outputs into aux
  sort(in,mid+1,right,aux,out);

  merge(aux,left,mid,right,out); // aux is input
}
```

- Non-recursive version?
  - Need to sort both child segments before sorting parent segment
  - Any child-before-parent order suffices
    - Recall: makeHeap
- Bottom-up ("reverse breadth-first" order) will work!

Merge Sort: Non-Recursive

```
void sort (std::vector<int>& in) {
  int N = in.size(); std::vector<int> aux(N);
  auto A = &in; auto B = &aux; // for conveniently switching between the two
  // imagine a nearly complete binary tree, where i-th node at height t
  // covers a block of T=2^t indices from i*T to (i+1)*T-1
  for(int t=0, T=1; T < N; T *= 2, ++t) { // merge pairs of T-long blocks
    for(int left=0; left < N; left += 2*T) {
      int mid = std::min(N-1,left+T-1), right = std::min(N-1,left+2*T-1);
      merge(*A,*B,left,mid,right); // if empty right half, just copies A to B
    }
   std::swap(A,B); // merge's output (B) should become its input (A) next time
  }
  // sorted array is in A
  if(A != &in) in = aux; // if necessary, copy aux to in
}
```

O(1) option available:
`in = std::move(aux);`

Attendance Break!

Login using CSE LDAP id at

https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

The selected students should come to the front

If you are not in the classroom, a report will be sent to DADAC!

Quicksort

- Another divide-and-conquer sorting algorithm
  - Almost in-place. Not stable.
- Divide into two lists, S_1 = [items < pivot] and S_2 = [items > pivot]
- Combine: [quicksort(S_1) [items == pivot] quicksort(S_2)]
- How is pivot picked? Various approaches
  - Pick the first element: Worst case $O(n^2)$
  - Pick a random element: Worst case O(n log n) with high probability
  - Pick the "median of medians": Worst case O(n log n)
  - Heuristic choices: Worst case $O(n^2)$, but often better than that
- Hybrid models: switch to another sorting if taking too long

E.g., when the data is sorted or reverse-sorted